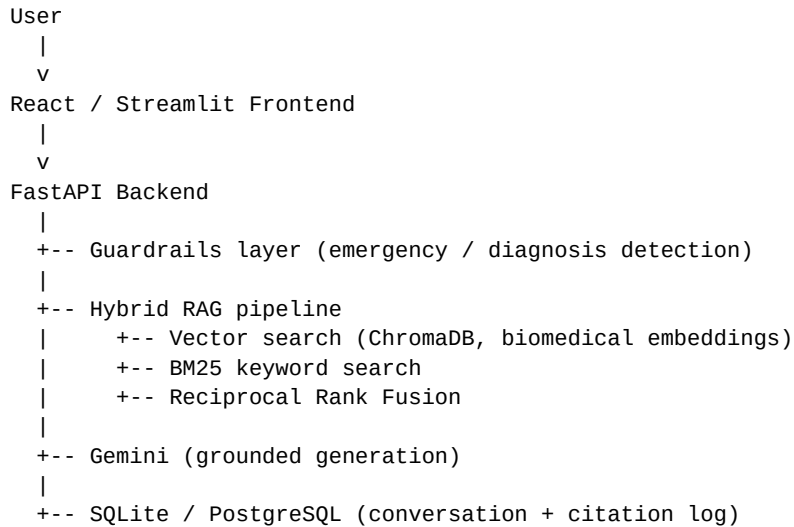


Medical RAG Assistant

A retrieval-augmented chatbot for clinical Q&A;, grounded in WHO/CDC guidelines with hybrid retrieval, source citations, and safety guardrails.

Architecture



Key design decisions

Hybrid retrieval, not vector search alone

Pure semantic search misses exact terms clinicians search for — drug names, dosages, ICD codes. BM25 catches lexical matches; dense retrieval catches semantic ones. Results are combined with Reciprocal Rank Fusion, avoiding the need to normalize two very different score scales.

Rule-based guardrails, not an extra LLM call

An LLM-based safety check adds latency, cost, and a second point of failure. A small set of explainable checks catches the clearest cases (emergency symptoms, personal diagnosis requests) fast and cheaply, and every flag is auditable.

Every answer traceable to source + page number

For medical content, an ungrounded answer is a liability. Every retrieved chunk that contributed to an answer is logged with its source document, page number, and relevance score, and surfaced to the user.

SQLite for dev, Postgres for prod

Zero-setup local development, with a straightforward swap to Postgres via a single environment variable for concurrent writes and real backups in production.

Evaluation methodology

Measured on a held-out set of clinical Q&A pairs using RAGAS plus a custom retrieval-precision check. Run `python -m eval.evaluate` against a live backend to populate real numbers.

Metric	What it measures	Result
Retrieval precision@k	Expected source retrieved in top-k	run eval.evaluate
Faithfulness (RAGAS)	Answer doesn't contradict retrieved context	run eval.evaluate
Answer relevancy (RAGAS)	Answer actually addresses the question	run eval.evaluate
Latency p50 / p95	End-to-end response time	run eval.evaluate

Project structure

```
medical-rag-bot/
+-- backend/
|   +-- main.py           FastAPI app, /chat endpoint
|   +-- rag/
|       +-- ingest.py     PDF -> chunks -> embeddings -> ChromaDB
|       +-- retrieve.py   hybrid vector + BM25 retrieval
|       +-- guardrails.py safety layer
|   +-- db/models.py      SQLAlchemy models
|   +-- eval/             RAGAS evaluation harness
|   +-- tests/            pytest unit tests
|   +-- Dockerfile
+-- frontend/
|   +-- app.py            Streamlit chat UI
|   +-- Dockerfile
+-- docker-compose.yml
+-- .github/workflows/ci.yml lint + test + build on push
```

Tradeoffs and future work

- BM25 index is in-memory, rebuilt at backend startup — fine to a few hundred thousand chunks, would move to Elasticsearch/OpenSearch beyond that.
- Guardrails are rule-based: fast and explainable but will miss paraphrased emergency language. A fine-tuned classifier would generalize better at the cost of latency.
- No authentication or rate limiting yet — required before any real deployment handling patient-adjacent queries.
- Embedding model is a general biomedical model; fine-tuning on the specific document corpus would likely improve retrieval precision further.

Full source, setup instructions, and CI configuration available in the project repository.